First read "1. Gradient Boosting Regression Theory.pdf"(must) and "2. Gradient Boosting Regression Math.pdf" from Theory folder.

Training

The training dataset we'll be working with (Binary Classification):

X		Y
iq	cg	placed
10	3.9	0
30	3.2	1
20	3.7	1
35	2.7	0
40	3.0	1

Also assume we've 2 models(Base Model + 2 Tree Models).

→ **Base Model**

———————

The Base model quantifies the log(odds) of the output column, placed. This log is $\log_e$. log and $\log_e$ are the same.

log(number of 1s / number of 0s) = log(3/2) = 0.405465.

Initially all rows are assigned with 0.405465 and in Regression we assigned with mean.

—X—		—Y—	Base Model
iq	cg	placed	pred_placed_0 (log odds)
10	3.9	0	0.405465
30	3.2	1	0.405465
20	3.7	1	0.405465
35	2.7	0	0.405465
40	3.0	1	0.405465

Now we'll quantify **Residual Error (Actual Output - Predicted_probability_ output)**. But our Predicted Output, pred_placed_0(log odds) is not a probability value but log value. So we need to convert the Log value into Probability :

$$\text{Probability} = \frac{1}{1 + e^{-\log(odds)}} = 1 / (1 + e^{-0.405465}) = 0.6$$

If the Probability is < 0.5, that means 0. If it's > 0.5, then 1. We got 0.6, but that doesn't mean we assign 1 to all rows. We need the probability 0.6 and it was not explained why in the tutorial. Most probably if it was 0.4, then we would get 0 and adding/multiplying with 0 is 0 which 0.4 prevents from happening.

—X—		—Y—	Base Model	
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)
10	3.9	0	0.405465	0.6
30	3.2	1	0.405465	0.6
20	3.7	1	0.405465	0.6
35	2.7	0	0.405465	0.6
40	3.0	1	0.405465	0.6

Now prediction is done by Base model and we'll calculate **Residual Error (Actual Output - Predicted_probability_output)** like we did in Regression :

—X—		—Y—	Base Model		
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)	res_0 (prob..)
10	3.9	0	0.405465	0.6	-0.6
30	3.2	1	0.405465	0.6	0.4
20	3.7	1	0.405465	0.6	0.4
35	2.7	0	0.405465	0.6	-0.6
40	3.0	1	0.405465	0.6	0.4

→ **First Tree Model**

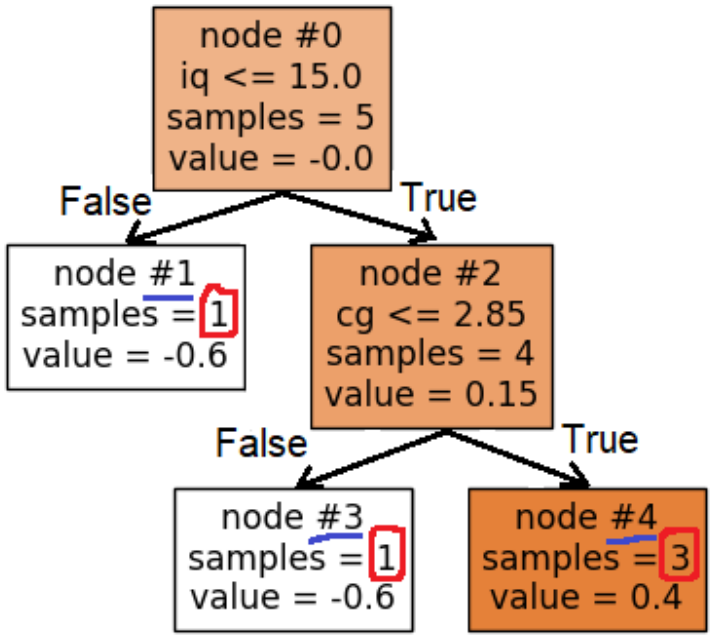
We train the first tree model (max_leaf_nodes = 3 to stop it from becoming Fully Grown Tree) with X and "**red_0(prob..)**" since each tree model is trained and predicted on the Previous Model's Error. The tree we'll use is DecisionTreeRegressor because y = "**red_0(prob..)**" is a continuous output, not binary or multi-classification. This model predicts the **Residual Error** of its previous model, Base Model :

—X—		—Y—	Base Model			Model 1
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)	res_0 (prob..)	pred_res_1 (probability)
10	3.9	0	0.405465	0.6	-0.6	-0.6
30	3.2	1	0.405465	0.6	0.4	0.4
20	3.7	1	0.405465	0.6	0.4	0.4
35	2.7	0	0.405465	0.6	-0.6	-0.6
40	3.0	1	0.405465	0.6	0.4	0.4

What if Model 1 predicted on the actual output column, placed? Then pred_placed_1 would be "whatever Base Model quantified|returned|predicted + what Model 1 predicted".

Base Model quantified pred_placed_0(log odds) which we converted into Probability value but it actually quantified the “log(odds)” value. We can’t add “log odds” value with Probability Value returned. We need to convert Model 1’s Predicted Residual Error’s Probability Value into “Predicted Residual Error’s log odds” value.

The Tree of Model 1 :



Since Model 1 predicts Residual Error (probability), so “value” represents “Predicted Residual Error(probability)”. The leaves node’s(1, 3, 4) values are the “pred_res_1(probability)” column’s values. If we change 4th Leaf’s value, we’re changing 3 row’s values since sample = 3.

To calculate the Residual Error(to train model) we need Probability value and for Prediction on the Output column we need Log(odds) value. After training the tree on Residual Error, of course we want to quantify the prediction of

this model on the output column. So we need a Log(odds) value for each leaf node. The equation: (PM = Previous Model)

$$\text{Log(odds) value for each Leaf Node "value"} = \frac{\Sigma \text{ PM's Residual}}{\Sigma \text{ [PM's predicted_output * (1 - PM's predicted_output)]}}$$

This tree visualization is just for convenience. Our actual dataset is the previous page's table. So how can we know which row of X belongs to which Leaf Node? Using **tree.apply(X)** and it'll return the leaves node's ID | Index.

—X—		—Y—	Base Model			Model 1
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)	res_0 (prob..)	enters_to_ leaf_node_1
10	3.9	0	0.405465	0.6	-0.6	1
30	3.2	1	0.405465	0.6	0.4	4
20	3.7	1	0.405465	0.6	0.4	4
35	2.7	0	0.405465	0.6	-0.6	3
40	3.0	1	0.405465	0.6	0.4	4

For leaf node 4, the Log(odds) value : (PM = Previous Model)
(PM's Residual =res_0(probability),
PM's predicted_output = pred_placed_0(probability),
Σ means i = 1 to count_of_rows|samples_in_leaf_node)

$$= \frac{\Sigma \text{ PM's Residual}}{\Sigma \text{ [PM's predicted_output * (1 - PM's predicted_output)]}}$$

$$= \frac{\Sigma (0.4 + 0.4 + 0.4)}{\Sigma [0.6 * (1-0.6) + 0.6 * (1-0.6) + 0.6 * (1-0.6)]}$$

$$= 1.2 / 0.72$$

$$= 1.67$$

For leaf node 1, the Log(odds) value : (Node 1 and 3 has same values)

$$= \frac{-0.6}{0.6 * (1 - 0.6)} = -0.25$$

Note : You may have thought, if probability = 1 / (1 + $e^{-\log odds}$), then Log odds = -log(1 - 1/probability). So why not directly convert the Leaf Node Value(probability) into Log odds using this equation? I tried and got Nan for positive values which we don't want e.g., -log(2.8) = NaN.

—X—		—Y—	Base Model			Model 1	
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)	res_0 (prob..)	enters_to_ leaf_node_1	pred_res_1 (log odds)
10	3.9	0	0.405465	0.6	-0.6	1	-2.50
30	3.2	1	0.405465	0.6	0.4	4	1.67
20	3.7	1	0.405465	0.6	0.4	4	1.67
35	2.7	0	0.405465	0.6	-0.6	3	-2.50
40	3.0	1	0.405465	0.6	0.4	4	1.67

Now we'll calculate the Prediction of Model 1 on Output column, Placed, **pred_placed_1(log odds)** :

pred_placed_0(log odds) + (learning_rate * pred_res_1(log odds)

—X—		—Y—	Base Model			Model 1		
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)	res_0 (prob..)	enters_to_ leaf_node_1	pred_res_1 (log odds)	pred_placed_1 (log odds)
10	3.9	0	0.405465	0.6	-0.6	1	-2.50	0.155
30	3.2	1	0.405465	0.6	0.4	4	1.67	0.572
20	3.7	1	0.405465	0.6	0.4	4	1.67	0.572
35	2.7	0	0.405465	0.6	-0.6	3	-2.50	0.155
40	3.0	1	0.405465	0.6	0.4	4	1.67	0.572

Now we calculate Residual Error of Model 1 which is :

= Actual Output - Predicted Output

= **placed - pred_placed_1(log odds)**

But as we've seen in the Base Model that to calculate Residual Error, we needed to convert pred_placed_0(log odds) to Probability value because "placed" in Probability format, 0 / 1. Similarly here we need to convert "pred_placed_1(log odds)" to "pred_placed_1(probability)" using $1 / (1 + e^{-\log(odds)})$ and then quantify Residual Error for Model 1 :

—X—		—Y—	Model 1				
iq	cg	placed	enters_to_ leaf_node_1	pred_res_1 (log odds)	pred_placed_1 (log odds)	pred_placed_1 (probabililty)	res_1 (prob..)
10	3.9	0	1	-2.50	0.155	0.538	-0.538
30	3.2	1	4	1.67	0.572	0.639	0.360
20	3.7	1	4	1.67	0.572	0.639	0.360
35	2.7	0	3	-2.50	0.155	0.538	-0.538
40	3.0	1	4	1.67	0.572	0.639	0.360

Finally we have after the Base Model and First Tree model trained :

—X—		—Y—	Base Model		
iq	cg	placed	pred_placed_0 (log odds)	pred_placed_0 (probability)	res_0 (prob..)
10	3.9	0	0.405465	0.6	-0.6
30	3.2	1	0.405465	0.6	0.4
20	3.7	1	0.405465	0.6	0.4
35	2.7	0	0.405465	0.6	-0.6
40	3.0	1	0.405465	0.6	0.4

—X—		—Y—	Model 1				
iq	cg	placed	enters_to_ leaf_node_1	pred_res_1 (log odds)	pred_placed_1 (log odds)	pred_placed_1 (probabililty)	res_1 (prob..)
10	3.9	0	1	-2.50	0.155	0.538	-0.538
30	3.2	1	4	1.67	0.572	0.639	0.360
20	3.7	1	4	1.67	0.572	0.639	0.360
35	2.7	0	3	-2.50	0.155	0.538	-0.538
40	3.0	1	4	1.67	0.572	0.639	0.360

pred_placed_m(log odds) =

$$\text{pred_placed_0(log odds)} + \sum_{i=1}^{m} (\text{learning_rate} * \text{pred_res_i(log odds)})$$

→ **Second Tree Model**

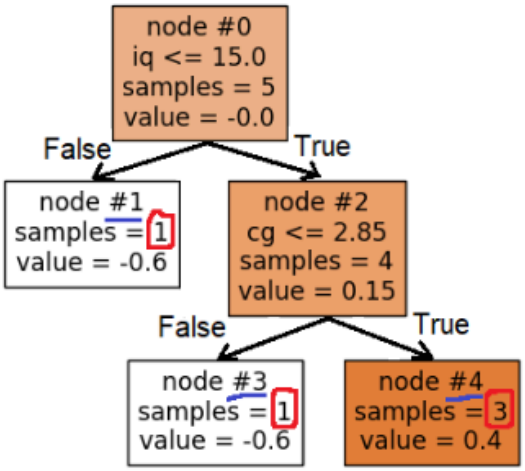
We train a DecisionTreeRegressor on X and Previous Model's Error with parameters so that the Tree can't grow fully because in Boosting we add Weak Trees together to create one strong tree, res_1(probability). After training the tree, we need to quantify the Log(odds) values on the Residual Error of each Leaf Node. Basically we'll repeat every process of the First Tree Model for this model. If we have m models, then we repeat the processes of the First Tree Model m times.

Testing/Predicting

In boosting the prediction happens by “You give your test dataset to each model and each model returns ‘something’. Then you add all the ‘something’ and that is the prediction result of the entire boosting algorithm”.

Here during the training how did we predict for each row ?

$$\text{pred_placed_0(log odds)} + \sum_{i=1}^{m} (\text{learning_rate} * \text{pred_res_i(log odds)})$$



X		Y	Model 1	
iq	cg	placed	enters_to_ leaf_node_1	pred_res_1 (log odds)
10	3.9	0	1	-2.50
30	3.2	1	4	1.67
20	3.7	1	4	1.67
35	2.7	0	3	-2.50
40	3.0	1	4	1.67

For each tree model, e.g. for Model 1 we didn’t care what it predicted(Residual Error in Probability) as we needed Log(odds) version value for each leaf node, we quantified i) the Leaves Node where each row of X went, ii) the predicted residual error (log odds).

Now let's say for a new point, iq = 12, cg = 3.3, in Model 1 it falls in Leaf Node 1 whose corresponding residual_error_log(odds) value is -2.50. In Model 2 it falls in Leaf Node 1 whose corresponding residual_error_log(odds) value is -2.16. So the prediction is :

= 0.40 + (0.1 * -2.50) + (0.1 * -2.16)
= -0.066 (log odds) [need to convert it into Probability]

= 0.51 (probability) [1 / (1 + $e^{-(-0.066)}$)]

This is how we'll predict in testing :

1) Take any unseen|testing dataset,

2) Find the leaves they fall into by tree.apply(x_test),

3) Collect their corresponding residual_error_log(odds) values. And to do that we need to collect each model's (leaves node, corresponding residual_error_log(odds) values) as a dictionary as far as I can conclude. If we've the dictionary, we can easily access the corresponding residual_error_log(odds) value of a Lead Node,

4) Now apply the pred_y_0(log odds) + Σ (learning_rate * pred_res_i(log odds)) formula for each row of x_test,

5) Convert each Predicted_y_log(odds) into Predicted_y_probability.